University of Nizwa

Object-Oriented Programming with Python

Assignment 1 — Object-Oriented Mini-Project

Coverage: Lectures 0–5 (Foundations → Encapsulation & Class Tools) **Weight:** 10% of course

Type: Individual, take-home **Due:** End of Week 6 **Submission:** Git repo named oop-assignment1

---

1. Overview

In this assignment you will design and build a small object-oriented program: a simple store inventory system. The goal is to practise everything from the first five lectures — writing classes, managing instance and class state, protecting data with properties and validation, and adding class-level tools such as alternative constructors, static helpers and factories. You should write clean, readable Python 3 and prove your code works with a short demonstration script.

2. Learning outcomes assessed

- **CLO1 —** Write correct, well-structured Python 3 using the core data types, control flow and functions.
- **CLO2 —** Design and implement classes that model entities as objects with attributes, methods and state.
- **CLO3 —** Apply encapsulation with properties and validation, and class tools such as classmethods, staticmethods and factories.

3. The scenario

“MiniMart” needs a small program to keep track of the products it stocks. Each product has a name, a SKU (stock-keeping code), a unit price and a quantity on hand. The store wants to know the value of each line, the total value of the inventory, and which products are running low.

4. Requirements

Part 1 — Core classes (CLO2)

1. Create a Product class. Its __init__ should accept name, sku, price and quantity.
2. Give Product a clear __repr__ (for example, Product('Milk', 'MLK001', 1.25, 12)).
3. Create an Inventory class that stores many products, keyed by their SKU (use a dictionary).
4. Add Inventory methods: add(product), remove(sku), find(sku) and total_value().

Part 2 — Encapsulation & validation (CLO3 · Lecture 4)

1. Turn price into a property that rejects non-numbers (TypeError) and negative values (ValueError).

2. Turn quantity into a property that accepts only non-negative integers.
3. Add a read-only computed property line_value that returns price × quantity.
4. Add a read-only property is_low_stock that is True when quantity is below 5.

Part 3 — Class tools & factories (CLO3 · Lecture 5)

1. Add a classmethod Product.from_csv_line(cls, line) that builds a Product from a string like "Milk,MLK001,1.25,12".
2. Add a staticmethod Product.is_valid_sku(sku) that returns True only for codes of three letters followed by three digits (e.g., MLK001).
3. Track how many Product objects have been created using a class attribute, exposed through a classmethod Product.count().
4. Add a classmethod Inventory.from_lines(cls, lines) that builds a complete inventory from a list of CSV lines.

Part 4 — Demonstration script (CLO1–CLO2)

1. In a main() function, build an inventory from the sample data below using your factory.
2. Print a neat report (use f-strings): each product with its line value, the total inventory value, and a list of low-stock items.
3. Guard the call to main() with if __name__ == "__main__":.

Sample data (you may copy this):

```
SAMPLE = [
    "Milk,MLK001,1.25,12",
    "Bread,BRD002,0.90,3",
    "Eggs,EGG003,2.40,30",
    "Coffee,COF004,7.50,2",
]
```

Part 5 — Bonus (optional, +5 marks)

- Write a few pytest tests that prove your validation works (e.g., a negative price raises ValueError).

5. Constraints

- Use Python 3.10+ and the standard library only (no third-party packages).
- Follow PEP 8 naming and layout; keep functions and methods small and focused.
- Per-instance data must be created in __init__, never as shared class attributes.
- Validate inputs where the requirements ask; never let an invalid Product exist.

6. Deliverables

- **product.py —** the Product class.
- **inventory.py —** the Inventory class.
- **main.py —** the demonstration script.
- **README.md —** how to run the program, and a short note on your design decisions.

- **test_*.py —** optional tests (bonus).

7. Marking rubric

Criterion	Marks
Part 1 — Core classes & structure (Product, Inventory)	25
Part 2 — Encapsulation & validation (properties, computed values)	25
Part 3 — Class tools & factories (classmethods, staticmethod, count)	20
Part 4 — Demonstration script & correct, well-formatted output	15
Code quality — PEP 8, readability, naming, README	15
Bonus — working pytest tests	+5
Total	**100**

8. Academic integrity

This is an individual assignment. You may discuss ideas, but the code you submit must be your own. If you use any AI assistance, you remain fully responsible for understanding and being able to explain every line; be prepared to walk through your code. Plagiarism and unattributed copying are handled under the Student Academic Misconduct Policy.